Avneet kaur

1024170050

# Assignment-3 UCS420 “Cognitive Computing”

Q.1 Create a dataset as follow in the table.

| Tid | Refund | Marital Status | Taxable Income | Cheat |
|---|---|---|---|---|
| 1 | Yes | Single | 125K | No |
| 2 | No | Married | 100K | No |
| 3 | No | Single | 70K | No |
| 4 | Yes | Married | 120K | No |
| 5 | No | Divorced | 95K | Yes |
| 6 | No | Married | 60K | No |
| 7 | Yes | Divorced | 220K | No |
| 8 | No | Single | 85K | Yes |
| 9 | No | Married | 75K | No |
| 10 | No | Single | 90K | Yes |

[2]:
```
import pandas as pd

data = {
    'Tid': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Refund': ['Yes', 'No', 'No', 'Yes', 'No',
               'No', 'Yes', 'No', 'No', 'No'],
    'Marital Status': ['Single', 'Married', 'Single', 'Married',
                       'Divorced', 'Married', 'Divorced',
                       'Single', 'Married', 'Single'],
    'Taxable Income': ['125K', '100K', '70K', '120K', '95K',
                       '60K', '220K', '85K', '75K', '90K'],
    'Cheat': ['No', 'No', 'No', 'No', 'Yes',
              'No', 'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)
print(df)
```

```
   Tid Refund Marital Status Taxable Income Cheat
0    1    Yes         Single           125K    No
1    2     No        Married           100K    No
2    3     No         Single            70K    No
3    4    Yes        Married           120K    No
4    5     No       Divorced            95K   Yes
5    6     No        Married            60K    No
6    7    Yes       Divorced           220K    No
7    8     No         Single            85K   Yes
8    9     No        Married            75K    No
9   10     No         Single            90K   Yes
```

Q.2 From the above table that you have created, locate row 0, 4, 7 and 8 using DataFrame.

```
3]:
print(df.iloc[[0, 4, 7, 8]])
```

| | Tid | Refund | Marital Status | Taxable Income | Cheat |
|---|---|---|---|---|---|
| 0 | 1 | Yes | Single | 125K | No |
| 4 | 5 | No | Divorced | 95K | Yes |
| 7 | 8 | No | Single | 85K | Yes |
| 8 | 9 | No | Married | 75K | No |

Q.3 Navigate the DataFrame and do the following task for the table created in question 1:

1. Select row from index 3 to 7.

```
[5]: # 1. Select rows from index 3 to 7
print(df.iloc[3:8])
```

| | Tid | Refund | Marital Status | Taxable Income | Cheat |
|---|---|---|---|---|---|
| 3 | 4 | Yes | Married | 120K | No |
| 4 | 5 | No | Divorced | 95K | Yes |
| 5 | 6 | No | Married | 60K | No |
| 6 | 7 | Yes | Divorced | 220K | No |
| 7 | 8 | No | Single | 85K | Yes |

2. Select row from index 4 to 8, and column 2 to 4.

```
[6]: # 2. Select rows from index 4 to 8 and columns 2 to 4
print(df.iloc[4:9, 2:5])
```

| | Marital Status | Taxable Income | Cheat |
|---|---|---|---|
| 4 | Divorced | 95K | Yes |
| 5 | Married | 60K | No |
| 6 | Divorced | 220K | No |
| 7 | Single | 85K | Yes |
| 8 | Married | 75K | No |

3. Select all rows with column index 1 to 3 (include index 3 during selection).

```
[7]: # 3. Select all rows with column index 1 to 3, including index 3
print(df.iloc[:, 1:4])
```

| | Refund | Marital Status | Taxable Income |
|---|---|---|---|
| 0 | Yes | Single | 125K |
| 1 | No | Married | 100K |
| 2 | No | Single | 70K |
| 3 | Yes | Married | 120K |
| 4 | No | Divorced | 95K |
| 5 | No | Married | 60K |
| 6 | Yes | Divorced | 220K |
| 7 | No | Single | 85K |
| 8 | No | Married | 75K |
| 9 | No | Single | 90K |

Q.4 Read a csv file and display its first five rows.
Note-: Download dataset from https://www.kaggle.com/datasets/uciml/iris)

[8]:
```
import pandas as pd

df = pd.read_csv('Iris.csv')
print(df.head())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

Q.5 From the csv file (uploaded in the Q.4) delete row 4, and delete column 3. Display the result.

[11]:
```
import pandas as pd

df = pd.read_csv('Iris.csv')

# Delete row with index 4
df = df.drop(index=4)

# Delete column with index 3
df = df.drop(columns=df.columns[3])

# Display the result
print(df)
```

```
      Id  SepalLengthCm  SepalWidthCm  PetalWidthCm         Species
0      1            5.1           3.5           0.2     Iris-setosa
1      2            4.9           3.0           0.2     Iris-setosa
2      3            4.7           3.2           0.2     Iris-setosa
3      4            4.6           3.1           0.2     Iris-setosa
5      6            5.4           3.9           0.4     Iris-setosa
..   ...            ...           ...           ...             ...
145  146            6.7           3.0           2.3  Iris-virginica
146  147            6.3           2.5           1.9  Iris-virginica
147  148            6.5           3.0           2.0  Iris-virginica
148  149            6.2           3.4           2.3  Iris-virginica
149  150            5.9           3.0           1.8  Iris-virginica

[149 rows x 5 columns]
```

Q.6 Create a sample dataset (employees.csv) containing information about employees in a company.

| Employee_ID | Name | Department | Age | Salary | Years_of_Experience | Joining_Date | Gender | Bonus | Rating |
|---|---|---|---|---|---|---|---|---|---|
| 101 | Alice | HR | 29 | 50000 | 4 | 2020-03-15 | Female | 5000 | 4.5 |
| 102 | Bob | IT | 34 | 70000 | 8 | 2017-07-19 | Male | 7000 | 4.0 |
| 103 | Charlie | IT | 41 | 65000 | 10 | 2013-06-01 | Male | 6000 | 3.8 |
| 104 | Diana | Marketing | 28 | 55000 | 3 | 2021-02-10 | Female | 4500 | 4.7 |
| 105 | Edward | Sales | 38 | 60000 | 12 | 2010-11-25 | Male | 5000 | 3.5 |

[12]:
```
import pandas as pd
data = {
    'Employee_ID': [101, 102, 103, 104, 105],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Edward'],
    'Department': ['HR', 'IT', 'IT', 'Marketing', 'Sales'],
    'Age': [29, 34, 41, 28, 38],
    'Salary': [50000, 70000, 65000, 55000, 60000],
    'Years_of_Experience': [4, 8, 10, 3, 12],
    'Joining_Date': ['2020-03-15', '2017-07-19',
                     '2013-06-01', '2021-02-10', '2010-11-25'],
    'Gender': ['Female', 'Male', 'Male', 'Female', 'Male'],
    'Bonus': [5000, 7000, 6000, 4500, 5000],
    'Rating': [4.5, 4.0, 3.8, 4.7, 3.5]
}
df = pd.DataFrame(data)

df.to_csv('employees.csv', index=False)

print(df)
```

```
   Employee_ID     Name Department  Age  Salary  Years_of_Experience  \
0          101    Alice         HR   29   50000                    4
1          102      Bob         IT   34   70000                    8
2          103  Charlie         IT   41   65000                   10
3          104    Diana  Marketing   28   55000                    3
4          105   Edward      Sales   38   60000                   12

  Joining_Date  Gender  Bonus  Rating
0   2020-03-15  Female   5000     4.5
1   2017-07-19    Male   7000     4.0
2   2013-06-01    Male   6000     3.8
3   2021-02-10  Female   4500     4.7
4   2010-11-25    Male   5000     3.5
```

WAP to determine the following using dataset in Q6:

a) Shape (number of rows and columns) of the DataFrame.

```
13]: import pandas as pd

df = pd.read_csv('employees.csv')

print("Shape of DataFrame:", df.shape)
```

```
Shape of DataFrame: (5, 10)
```

b) Summary of the DataFrame that includes the data types and non-null counts for each column.

```
[14]: import pandas as pd

df = pd.read_csv('employees.csv')

df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 10 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Employee_ID          5 non-null      int64  
 1   Name                 5 non-null      str    
 2   Department           5 non-null      str    
 3   Age                  5 non-null      int64  
 4   Salary               5 non-null      int64  
 5   Years_of_Experience  5 non-null      int64  
 6   Joining_Date         5 non-null      str    
 7   Gender               5 non-null      str    
 8   Bonus                5 non-null      int64  
 9   Rating               5 non-null      float64
dtypes: float64(1), int64(5), str(4)
memory usage: 392.0 bytes
```

c) Generate descriptive statistics.

```
[16]: import pandas as pd

df = pd.read_csv('employees.csv')

print(df.describe())
```

```
       Employee_ID        Age       Salary  Years_of_Experience   Bonus
count     5.000000   5.000000      5.00000             5.000000     5.0
mean    103.000000  34.000000  60000.00000             7.400000  5500.0
std       1.581139   5.612486   7905.69415             3.847077  1000.0
min     101.000000  28.000000  50000.00000             3.000000  4500.0
25%     102.000000  29.000000  55000.00000             4.000000  5000.0
50%     103.000000  34.000000  60000.00000             8.000000  5000.0
75%     104.000000  38.000000  65000.00000            10.000000  6000.0
max     105.000000  41.000000  70000.00000            12.000000  7000.0

         Rating
count  5.000000
mean   4.100000
std    0.494975
min    3.500000
25%    3.800000
50%    4.000000
75%    4.500000
max    4.700000
```

d) Display the first 5 rows and last 3 rows of the dataset.

[17]:
```
import pandas as pd

# Read the employee dataset
df = pd.read_csv('employees.csv')

# Display first 5 rows
print("First 5 rows:")
print(df.head(5))

# Display last 3 rows
print("\nLast 3 rows:")
print(df.tail(3))
```

```
First 5 rows:
   Employee_ID     Name Department  Age  Salary  Years_of_Experience  \
0          101    Alice         HR   29   50000                    4
1          102      Bob         IT   34   70000                    8
2          103  Charlie         IT   41   65000                   10
3          104    Diana  Marketing   28   55000                    3
4          105   Edward      Sales   38   60000                   12

  Joining_Date  Gender  Bonus  Rating
0   2020-03-15  Female   5000     4.5
1   2017-07-19    Male   7000     4.0
2   2013-06-01    Male   6000     3.8
3   2021-02-10  Female   4500     4.7
4   2010-11-25    Male   5000     3.5

Last 3 rows:
   Employee_ID     Name Department  Age  Salary  Years_of_Experience  \
2          103  Charlie         IT   41   65000                   10
3          104    Diana  Marketing   28   55000                    3
4          105   Edward      Sales   38   60000                   12

  Joining_Date  Gender  Bonus  Rating
2   2013-06-01    Male   6000     3.8
3   2021-02-10  Female   4500     4.7
4   2010-11-25    Male   5000     3.5
```

e) Calculate the following statistics from the dataset:

i. The average salary of employees.

[18]:
```
print("Average Salary:", df['Salary'].mean())
```

```
Average Salary: 60000.0
```

ii. The total bonus paid to all employees.

[19]:
```
print("Total Bonus Paid:", df['Bonus'].sum())
```

```
Total Bonus Paid: 27500
```

iii. The youngest employee's age.

```
[20]: print("Youngest Employee's Age:", df['Age'].min())
```

```
Youngest Employee's Age: 28
```

iv. The highest performance rating.

```
[21]: print("Highest Performance Rating:", df['Rating'].max())
```

```
Highest Performance Rating: 4.7
```

f) Sort the DataFrame by the Salary column in descending order.

```
[22]: df_sorted = df.sort_values(by='Salary', ascending=False)
print(df_sorted)
```

```
   Employee_ID     Name Department  Age  Salary  Years_of_Experience  \
1          102      Bob         IT   34   70000                    8
2          103  Charlie         IT   41   65000                   10
4          105   Edward      Sales   38   60000                   12
3          104    Diana  Marketing   28   55000                    3
0          101    Alice         HR   29   50000                    4

  Joining_Date  Gender  Bonus  Rating
1   2017-07-19    Male   7000     4.0
2   2013-06-01    Male   6000     3.8
4   2010-11-25    Male   5000     3.5
3   2021-02-10  Female   4500     4.7
0   2020-03-15  Female   5000     4.5
```

g) Add a new column that categorizes employees based on their performance rating:

i. Excellent for ratings >= 4.5

```
[23]: df['Performance'] = df['Rating'].apply(
    lambda rating: 'Excellent' if rating >= 4.5 else ''
)
print(df[['Name', 'Rating', 'Performance']])
```

```
      Name  Rating Performance
0    Alice     4.5   Excellent
1      Bob     4.0
2  Charlie     3.8
3    Diana     4.7   Excellent
4   Edward     3.5
```

ii. Good for ratings >= 4.0 but < 4.5

```
[24]: df['Performance'] = df['Rating'].apply(
    lambda rating: 'Good' if 4.0 <= rating < 4.5 else ''
)
print(df[['Name', 'Rating', 'Performance']])
```

```
      Name  Rating Performance
0    Alice     4.5
1      Bob     4.0        Good
2  Charlie     3.8
3    Diana     4.7
4   Edward     3.5
```

iii. Average for ratings < 4.0

```
[25]: df['Performance'] = df['Rating'].apply(
          lambda rating: 'Average' if rating < 4.0 else '
      )
      print(df[['Name', 'Rating', 'Performance']])
```

```
      Name  Rating Performance
0    Alice     4.5
1      Bob     4.0
2  Charlie     3.8     Average
3    Diana     4.7
4   Edward     3.5     Average
```

h) Identify missing values in the DataFrame.

```
[26]: print(df.isnull().sum())
```

```
Employee_ID            0
Name                   0
Department             0
Age                    0
Salary                 0
Years_of_Experience    0
Joining_Date           0
Gender                 0
Bonus                  0
Rating                 0
Performance            0
dtype: int64
```

i) Rename the Employee_ID column to ID.

```
[28]: df = df.rename(columns={'Employee_ID': 'ID'})

      print(df)
```

```
    ID     Name Department  Age  Salary  Years_of_Experience Joining_Date
0  101    Alice         HR   29   50000                    4   2020-03-15
1  102      Bob         IT   34   70000                    8   2017-07-19
2  103  Charlie         IT   41   65000                   10   2013-06-01
3  104    Diana  Marketing   28   55000                    3   2021-02-10
4  105   Edward      Sales   38   60000                   12   2010-11-25

   Gender  Bonus  Rating Performance
0  Female   5000     4.5
1    Male   7000     4.0
2    Male   6000     3.8     Average
3  Female   4500     4.7
4    Male   5000     3.5     Average
```

j) Find all employees who:

i. Have more than 5 years of experience.

```
[29]: print(df[df['Years_of_Experience'] > 5])
```

```
    ID     Name Department  Age  Salary  Years_of_Experience Joining_Date
1  102      Bob         IT   34   70000                    8   2017-07-19
2  103  Charlie         IT   41   65000                   10   2013-06-01
4  105   Edward      Sales   38   60000                   12   2010-11-25

  Gender  Bonus  Rating Performance
1   Male   7000     4.0
2   Male   6000     3.8     Average
4   Male   5000     3.5     Average
```

ii. Belong to the IT department.

```
[30]: print(df[df['Department'] == 'IT'])
```

```
    ID     Name Department  Age  Salary  Years_of_Experience Joining_Date  \
1  102      Bob         IT   34   70000                    8   2017-07-19
2  103  Charlie         IT   41   65000                   10   2013-06-01

  Gender  Bonus  Rating Performance
1   Male   7000     4.0
2   Male   6000     3.8     Average
```

k) Modify the dataset by adding a new column, Tax, which deducts 10% of the Salary.

```
[31]: df['Tax'] = df['Salary'] * 0.10

print(df)
```

```
    ID     Name Department  Age  Salary  Years_of_Experience Joining_Date  \
0  101    Alice         HR   29   50000                    4   2020-03-15
1  102      Bob         IT   34   70000                    8   2017-07-19
2  103  Charlie         IT   41   65000                   10   2013-06-01
3  104    Diana  Marketing   28   55000                    3   2021-02-10
4  105   Edward      Sales   38   60000                   12   2010-11-25

   Gender  Bonus  Rating Performance     Tax
0  Female   5000     4.5              5000.0
1    Male   7000     4.0              7000.0
2    Male   6000     3.8     Average  6500.0
3  Female   4500     4.7              5500.0
4    Male   5000     3.5     Average  6000.0
```

l) Save the modified DataFrame (with added columns) to a new CSV file.

```
[32]: df.to_csv('employees_modified.csv', index=False)

print("Modified DataFrame saved successfully!")
```

```
Modified DataFrame saved successfully!
```